SPRECHSKRIPT

Gruber Login

Zum Ablesen · Gesamtdauer ca. 8–10 Minuten

Person A ca. 4–5 Min. · Person B ca. 4–5 Min. · *Kursiv* = *Regieanweisung* · Stand: 22. Oktober 2026

Teil 1 · Person A – Idee, Aufbau, Login-Modi

1. Begrüßung und Idee ca. 1 Min.

Guten Tag, wir stellen Ihnen unser Projekt „Gruber Login“ vor. Das ist eine Login-Anwendung, bei der man bei der Registrierung selbst wählt, wie sicher die Anmeldung sein soll.

Die Idee dahinter: Nicht jede Seite braucht die gleiche Sicherheit. Deshalb bieten wir drei Stufen an – vom einfachen Passwort bis zur Authenticator-App.

2. Technischer Aufbau ca. 1,5 Min.

Die Anwendung besteht aus drei Docker-Containern:

- **Datenbank:** MySQL speichert Benutzer, Sessions und die temporären Codes.
- **Web:** Das Backend mit FastAPI in Python liefert die Seiten aus und stellt die Schnittstellen bereit.
- **Cloudflared:** Ein Tunnel, der die Seite per HTTPS aus dem Internet erreichbar macht, ohne dass wir Ports am Router öffnen müssen.

Das Backend ist ein Python-Paket `app/` mit rund 1.400 Zeilen inklusive Kommentaren. Wichtig ist die Aufteilung: Die **Router** nehmen die HTTP-Anfragen an, zum Beispiel `login` oder `register`. Die **Services** enthalten die Regeln – wer darf sich anmelden, wie wird ein Code geprüft. Und kleine Module wie `security.py` oder `db.py` kümmern sich nur um Kryptografie oder die Datenbank. So steht in jeder Datei eine Sache, und keine Datei ist länger als etwa 150 Zeilen.

Alles startet mit einem einzigen Befehl: `docker compose up`.

3. Die drei Login-Modi ca. 2 Min.

Modus 1: Nur Passwort

E-Mail und Passwort, danach ist man direkt eingeloggt. Das ist der einfachste Modus.

Modus 2: E-Mail-Code

Nach dem Passwort bekommt man einen sechsstelligen Code per E-Mail. Er ist zehn Minuten gültig. Das ist eine Zwei-Faktor-Authentifizierung: etwas, das man weiß, plus etwas, das man besitzt – nämlich Zugriff auf das Postfach.

Modus 3: Authenticator-App

Bei der Registrierung wird ein QR-Code angezeigt, den man mit einer App wie Google Authenticator scannt. Den ersten Code aus der App bestätigt man direkt auf der Seite – damit ist die Einrichtung

fertig und man ist angemeldet. Danach braucht jeder Login den Code aus der App. Er ändert sich alle 30 Sekunden und funktioniert ohne Internet. Grundlage ist der Standard RFC 6238.

> *Übergabe: „Wie ein Login technisch abläuft und wie wir die Daten schützen, erklärt dir jetzt [Name Person B].“*

Teil 2 · Person B – Ablauf, Sicherheit, Demo, Fazit

4. Ablauf eines Logins ca. 1,5 Min.

1. Der Nutzer gibt E-Mail und Passwort ein. Der Browser schickt beides über HTTPS an den Server.
2. Der Server sucht das Konto und prüft das Passwort mit Argon2id. Das dauert absichtlich etwa 50 Millisekunden.
3. Je nach Modus ist man jetzt eingeloggt, oder der Server verlangt den zweiten Faktor: E-Mail-Code oder Authenticator-Code.
4. Nach erfolgreicher Prüfung erzeugt der Server eine Session: einen zufälligen Token in einem `HttpOnly`-Cookie, sieben Tage gültig. In der Datenbank liegt nur der Hash des Tokens.
5. Die Willkommensseite prüft bei jedem Aufruf diesen Cookie. Ohne gültige Session wird man zurück zum Login geleitet.

5. Sicherheit ca. 1,5 Min.

- **Passwörter** liegen nie im Klartext in der Datenbank, sondern nur als Argon2id-Hash mit zufälligem Salt. Aus dem Hash lässt sich das Passwort nicht zurückrechnen.
- **Warum wird nicht im Browser gehasht?** Ein Hash aus dem Browser wäre selbst wieder ein Passwort: Wer ihn abfängt, kann sich damit anmelden. Ein einzelnes SHA-256 wäre außerdem viel zu schnell berechnet. Deshalb rechnet der Server – geschützt durch HTTPS – und verwendet mit Argon2id ein absichtlich langsames, speicherresistentes Verfahren.
- **Der Cookie** ist `HttpOnly`, JavaScript kann ihn also nicht auslesen. `SameSite=Lax` blockiert fremde Formular-Posts, und standardmäßig wird er nur über HTTPS gesendet.
- **Codes in der Datenbank:** Der E-Mail-Code liegt nur als Signatur mit dem Server-Geheimnis, das TOTP-Secret verschlüsselt. Ein Datenbank-Dump zeigt also keinen gültigen Code.
- **Durchprobieren wird gebremst:** Nach fünf falschen Codes ist der Code ungültig. Dazu kommen Rate-Limits pro IP und pro E-Mail-Adresse. Ein TOTP-Code kann außerdem nur einmal verwendet werden – jeder 30-Sekunden-Schritt wird nur einmal akzeptiert.
- **Fremde Konten** lassen sich nicht überschreiben: Wer sich mit einer schon vergebenen E-Mail registriert, braucht das richtige Passwort.
- **Anti-Enumeration:** Beim Passwort-Reset verrät die Antwort nicht, ob eine E-Mail-Adresse existiert.

Ehrlich gesagt: Das sind unsere Grenzen

Das Projekt ist ein Lern- und Demoprojekt, kein fertiges Produktivsystem. Uns ist bewusst:

- Der Rate-Limiter liegt im Speicher und gilt pro Prozess. Für mehrere Worker bräuchten wir Redis.
- Im Entwicklungsmodus stehen die Codes im Klartext im Log. Für die Demo praktisch, für Produktion nicht gedacht.
- Datenbank-Änderungen passieren beim Start; für echte Migrationen wäre Alembic der nächste Schritt.
- Es gibt noch keine Wiederherstellungscodes für ein verlorenes Handy.

6. Live-Demo ca. 2 Min.

Vorher prüfen: Docker Compose läuft, http://localhost:5000 oder die Tunnel-Adresse ist offen, ein Log-Fenster (`docker compose logs -f web`) liegt bereit, die Authenticator-App auf dem Handy ist eingerichtet.

1. **Passwort:** Registrieren mit „Nur Passwort“ – man landet direkt auf der Willkommensseite.
2. **E-Mail-Code:** Abmelden, neu registrieren mit „Code per E-Mail“. Ist kein SMTP konfiguriert, erscheint der Code in einem gelben Kasten auf der Seite und zusätzlich im Log. Code eingeben – angemeldet.
3. **Authenticator:** Registrieren mit „Authenticator-App“, QR-Code mit dem Handy scannen, den angezeigten Code eingeben – die Einrichtung ist bestätigt und man ist direkt angemeldet.
4. **Passwort vergessen:** Abmelden, „Passwort vergessen?“ wählen, Code und neues Passwort eingeben, dann mit dem neuen Passwort anmelden.
5. **Schutz:** `/welcome` ohne Login aufrufen – man wird zur Startseite zurückgeleitet.

7. Fazit und Ausblick ca. 0,5 Min.

Wir haben gezeigt: eine Login-Anwendung mit drei Sicherheitsstufen, die Passwörter mit Argon2id geschützt speichert, Durchprobieren bremst, per Docker reproduzierbar startet und wenig Ressourcen braucht – etwa 0,5 bis 0,8 Gigabyte Arbeitsspeicher als Schätzwert. Dazu kommen 30 automatisierte Tests, die alle Abläufe ohne Datenbank und ohne Mailserver prüfen.

Als nächste Schritte würden wir den Rate-Limiter über Redis teilen, Datenbank-Migrationen mit Alembic einführen und Wiederherstellungscodes für die Authenticator-App ergänzen.

Vielen Dank! Haben Sie Fragen?

Mögliche Fragen und Kurzantworten

Frage	Kurzantwort
Warum Argon2id und nicht bcrypt?	Beides ist sicher. Argon2id ist die OWASP-Empfehlung für neue Projekte: Es ist speicherresistent, ein Angreifer kann also nicht einfach viele Hashes parallel berechnen.
Warum kein Hashen im Browser?	Der Hash wäre selbst ein gültiges Passwort („Pass-the-Hash“). Außerdem ist SHA-256 für Passwörter viel zu schnell. HTTPS plus Argon2id auf dem Server ist der übliche Weg.

Frage	Kurzantwort
Unterschied E-Mail-Code und Authenticator?	Der E-Mail-Code hängt vom Postfach ab. Der Authenticator-Code wird auf dem Handy aus einem geheimen Schlüssel und der Uhrzeit berechnet und funktioniert offline.
Sind sechs Ziffern nicht zu wenig?	Es gibt eine Million Kombinationen. Nach fünf Fehlversuchen ist der Code ungültig, zusätzlich greifen Rate-Limits. TOTP-Codes gelten nur 30 Sekunden und jeder Schritt nur einmal.
Wozu der Cloudflare Tunnel?	Er liefert HTTPS und Erreichbarkeit aus dem Internet, ohne Portfreigaben am Router.
Was passiert, wenn das Cookie gestohlen wird?	Dann hat man die Session. Deshalb `HttpOnly` und `Secure`, und Sessions laufen nach sieben Tagen ab. Ein Logout löscht sie sofort auf dem Server.
Was passiert ohne `APP_SECRET`?	Die App startet mit einer Warnung und einem zufälligen Geheimnis. Vorhandene TOTP-Secrets sind nach einem Neustart dann nicht mehr entschlüsselbar – deshalb gehört der Wert fest in die `.env`.
Was fehlt für den Produktivbetrieb?	Geteilter Rate-Limiter (Redis), Migrationen mit Alembic, E-Mail-Verifizierung bei der Registrierung, Wiederherstellungscodes und Monitoring.